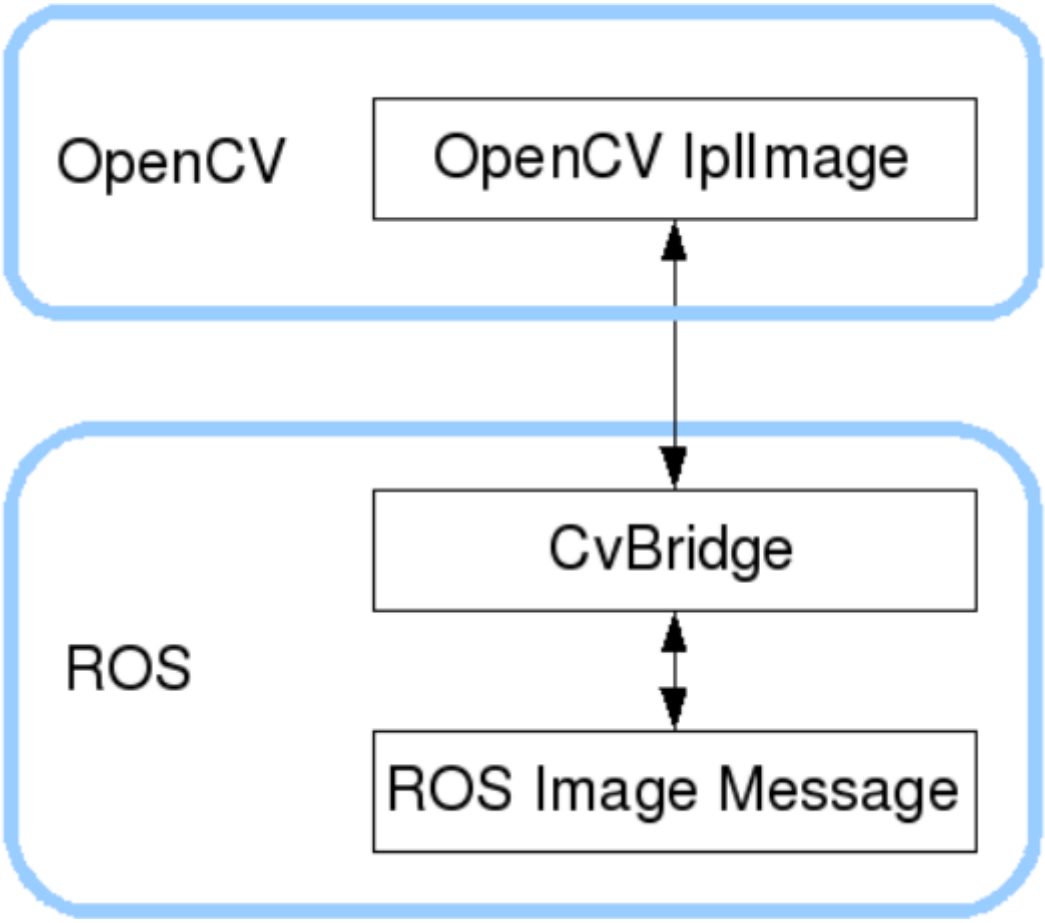
2. ROS + OpenCV Applications

ROS transmits images using its own proprietary `sensor_msgs/Image` message format, which cannot be directly processed by standard image processing libraries. However, the provided **CvBridge** tool enables seamless conversion of image data formats. **CvBridge** is a ROS library that effectively serves as a bridge between ROS and OpenCV.

The process for converting image data between OpenCV and ROS is illustrated in the diagram below:



1. Launching the Camera

Access the Docker container terminal and enter the following command:

```
ros2 launch dofbot_info camera_arm_kin.launch.py
```

```
# View the contents of the image topic data
ros2 topic echo /image_raw
```

Let's first capture a single frame of RGB color image data to inspect its details:

```
header:
  stamp:
    sec: 1682406733
    nanosec: 552769817
  frame_id: camera_color_optical_frame
height: 480
width: 640
encoding: rgb8
is_bigendian: 0
step: 1920
data:
- 156
- 130
- 139
- 158
- 132
- 141
- 160
- 134
- 145
- 161
```

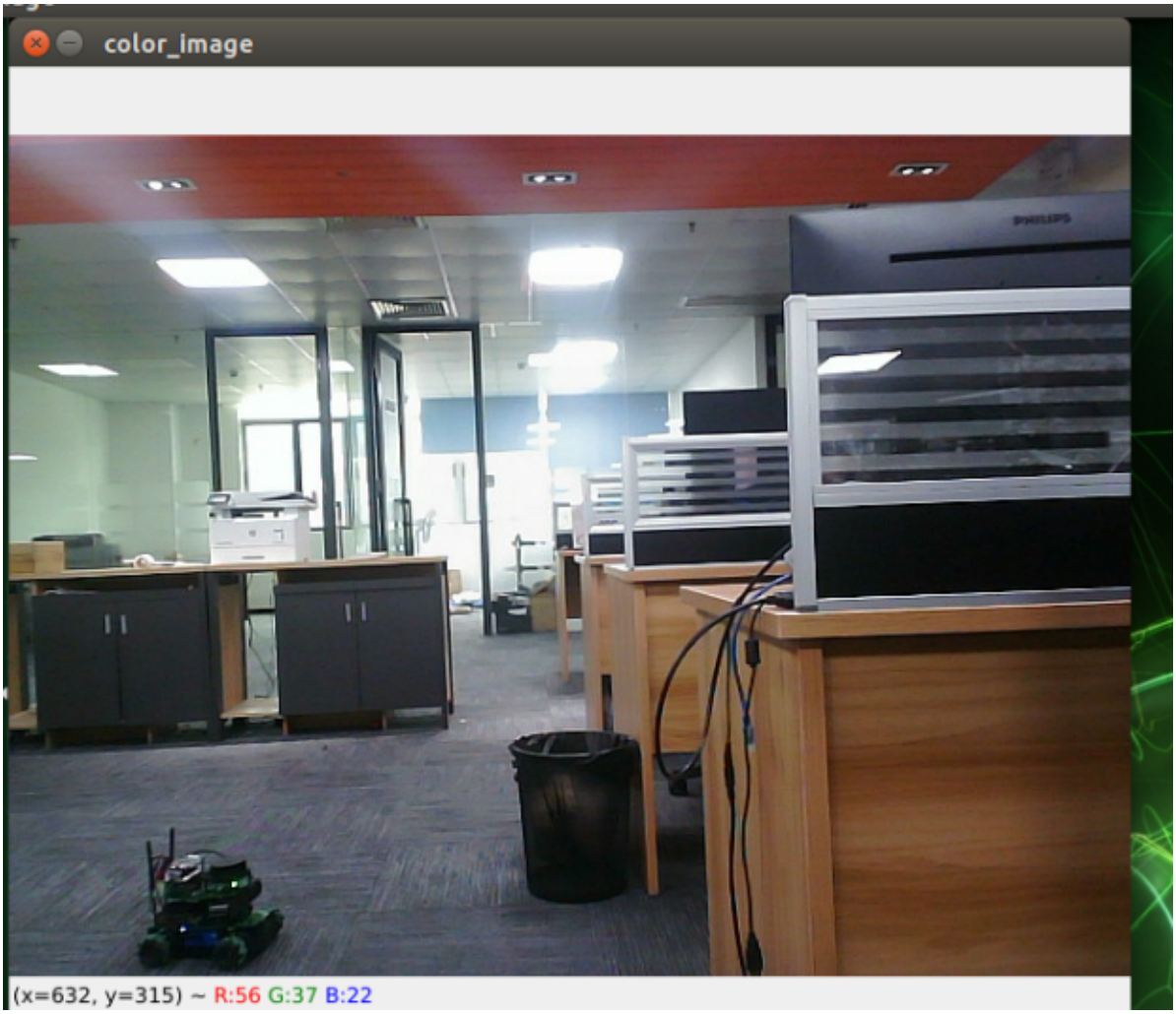
This output displays the basic information regarding the image. One particularly important value here is **encoding**; in this instance, the value is **rgb8**. This value indicates that the encoding format for this specific image frame is `rgb8`—a detail that will need to be referenced later when performing data conversions.

2. Subscribing to RGB Image Topics and Displaying the RGB Image

2.1. Running the Command

Access the Docker container terminal and enter:

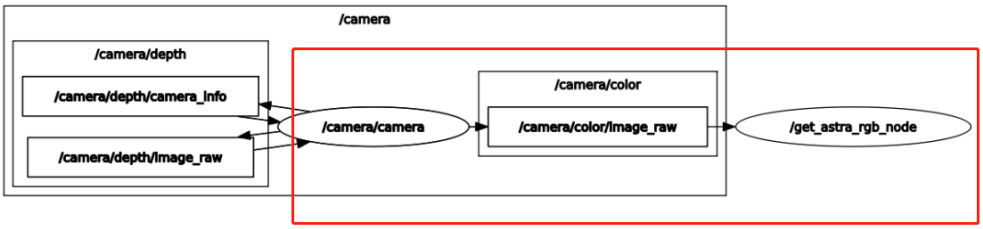
```
# RGB Color Image Display Node
ros2 run dofbot_pro_vision astra_rgb_image
```



2.2. Viewing the Node Communication Graph

Enter the following in the terminal:

```
ros2 run rqt_graph rqt_graph
```



2.3. Core Code Analysis

Path to the provided source code package:

```
/dofbot_pro_vision/dofbot_pro_vision/astra_rgb_image.py
```

As observed in Section 2.2, the `/get_astra_rgb_node` subscribes to the `/image_raw` topic; it then performs data conversion to transform the topic data into image data for display. The relevant code is as follows:

```
# Import the OpenCV and cv_bridge libraries
import cv2 as cv
from cv_bridge import CvBridge
# Create a CvBridge object
self.bridge = CvBridge()
# Define a subscriber to listen for the RGB color image topic data published by
the camera node
self.sub_img = self.create_subscription(Image, '/image_raw', self.handleTopic,
100)
# Convert the message (msg) into image data; 'bgr8' here specifies the image
encoding format
frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")
```

2. Subscribing to Image Data and Publishing the Converted Image Data

2.1. Running the Command

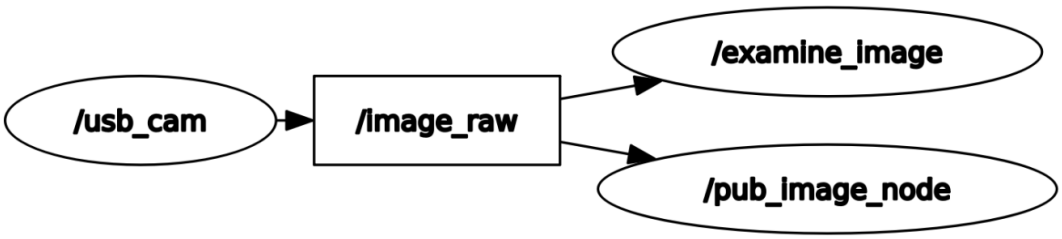
If you encounter an error, it is likely because a previously executed node program is still occupying the video device. In this case, you need to restart the device; restart and re-run the Docker container to resolve the issue.

```
# Enter this command after accessing the Docker container terminal
ros2 launch dofbot_info camera_arm_kin.launch.py
# Once inside the Docker container terminal, enter:
ros2 run dofbot_pro_vision pub_image
```

2.2. Viewing the Node Communication Graph

In the terminal, enter:

```
ros2 run rqt_graph rqt_graph
```



2.3. Viewing Topic Data

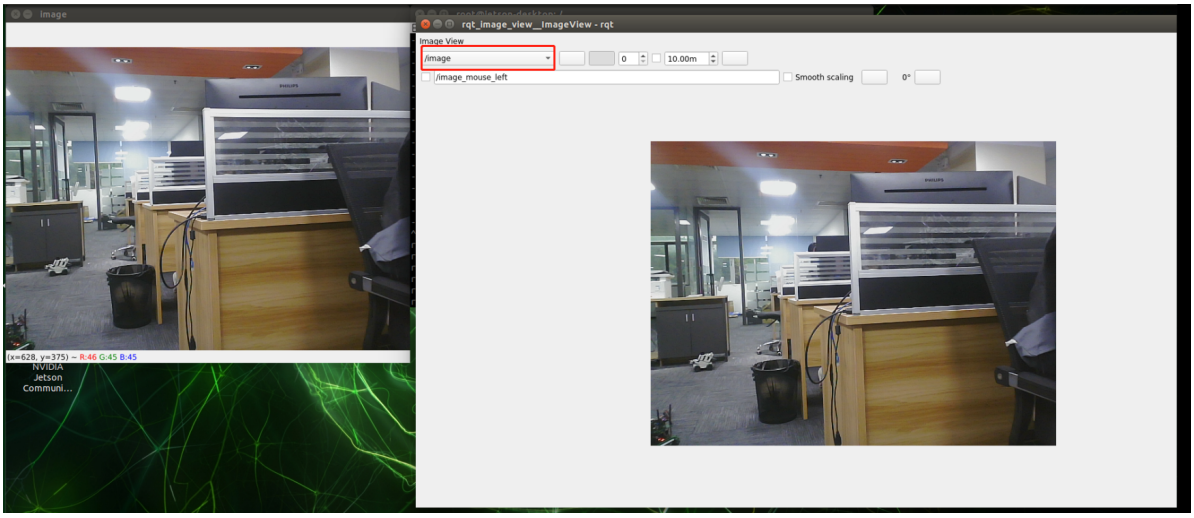
In the Docker terminal, enter:

```
ros2 topic echo /image
```

```
---
header:
  stamp:
    sec: 0
    nanosec: 0
  frame_id: ''
height: 480
width: 640
encoding: bgr8
is_bigendian: 0
step: 1920
data:
- 95
- 86
- 122
- 90
- 81
- 117
- 96
- 83
```

You can use the `rqt_image_view` tool to view the images:

```
ros2 run rqt_image_view rqt_image_view
```



Once opened, select the topic name `/image` in the upper-left corner to view the image stream.

2.4. Core Code Analysis

Code path:

```
~/dofbot_ws/src/dofbot_pro_vision/dofbot_pro_vision
```

The implementation steps are generally similar to the previous two examples: the program first subscribes to the `/image_raw` topic data and converts it into raw image data. However, in this instance, an additional format conversion is performed to transform the raw image data back into topic data, which is then published. In essence, the data flow is: Image Topic Data → Raw Image Data → Image Topic Data.

```python
# Import the OpenCV and CvBridge libraries
import cv2 as cv
from cv_bridge import CvBridge
# Create a CvBridge object
self.bridge = CvBridge()
# Define a subscriber to subscribe to the USB image topic data
self.sub_img = self.create_subscription(Image, 'image_raw', self.handleTopic,
500)
# Define a publisher for the image topic data
self.pub_img = self.create_publisher(Image, '/image', 500)
# Convert the message into image data using imgmsg_to_cv2; here, "bgr8" specifies
the image encoding format
frame = self.bridge.imgmsg_to_cv2(msg, "bgr8")
# Convert the image data back into an image topic message (using cv2_to_imgmsg)
and then publish it
msg = self.bridge.cv2_to_imgmsg(frame, "bgr8")
self.pub_img.publish(msg)
```